Willkommen in der Mikrocontroller.net Artikelsammlung. Alle Artikel hier können nach dem Wiki-Prinzip von jedem bearbeitet werden. Zur Hauptseite der Artikelsammlung

Antrax GPS/GSM/GPRS Shield

Aus der Mikrocontroller.net Artikelsammlung, mit Beiträgen verschiedener Autoren (siehe Versionsgeschichte)

von Stefan Schuster

Dieser Artikel nimmt am Artikelwettbewerb 2012/2013 teil.

GPS und GSM sind häufig mit Arduino verwendete Technologien. GPS erlaubt die Poisitionsbestimmung mit Hilfe von Satellitensignalen und GSM erlaubt die Kommunikation über das Mobilfunk-Netz. Mit GPRS können über das Mobilfunknetz Datenverbindungen zu Internet-Servern aufgebaut werden. Da die Positionsbestimmung meist outdoor und fernab eines zugänglichen WLAN etc. erfolgt, bleibt für die Verarbeitung der Positionsdaten oft nur der Weg, diese zu speichern und später zu verarbeiten. Das ist für viele Anwendungen nicht ideal. Bei der Kombination von GPS und einer Kommunikationsmöglichkeit über Mobilfunk können Anwendungs-Daten mit GPS Informationen kombiniert und per GSM verschickt werden. Damit werden neue Anwendungen denkbar, die auf der Online-Verarbeitung von Daten auf einem Server basieren. Beispielsweise eine Echtzeit-Verfolgung eines Arduino-Boards auf einer Webseite. Ebenso kann über GPRS eine Datenverbindung aufgebaut werden und von einem Server Informationen gelesen werden. Es ist beispielsweise denkbar, dass Informationen zu Positionen angefragt werden.

Dieser Artikel beschreibt die Verwendung des GPS/GSM Shields von Antrax Datentechnik mit einem Arduino Board. Es wird eine eigene Bibliothek für die Verwendung von GSM und GPS vorgestellt (frei zum Download). Die verwendeten Chips sind auch in vielen anderen Produkten vorhanden oder können direkt mit wenig Beschaltung verwendet werden. Daher kann dieser Artikel auch bei ähnlichen Produkten mit diesen Chips weiterhelfen. Anhand von Beispielen wird auf die Funktionen des Shields eingegangen. Getestet sind die Beispiele auf einem Arduino Mega 2560 R3. Die Bibliothek kann mit kleinen Anpassungen auch auf einem Arduino Uno verwendet werden. Als Kommunikationsmöglichkeiten werden sowohl SMS Versand als auch GPRS Datenverbindungen unterstützt.

Das Ziel des Artikels ist es, eine Grundlage für die Arbeit mit dem Shield oder den einzelnen sowie kompatiblen ICs zu liefern. Die Leser sollen verstehen wie die GPS und GSM Module mit dem Arduino kommunizieren. Ebenso soll ein grundlegendes Verständnis geschaffen werden, wie die Biliothek intern funktioniert, so dass bei Bedarf eigene Erweiterungen erstellt werden können. Beispielprogramme sollen die Verwendung zeigen, ggfs. auf verschiedenen Ebenen

(direkte Verarbeitung der Nachrichten der Module oder Verarbeitung durch Bibliotheks-Methoden). Es wird davon ausgegangen, dass der Leser mit der Arduino-Plattform und der IDE sowie dem darin integrierten Tool "Serial Monitor" vertraut ist.

Inhaltsverzeichnis

1 Die Hardware und Anbindung des GPS/GSM Shields
- 1.1 Verwendete Pins
- 1.2 GSM Teil
- 1.3 GPS Teil

2 Die Software Bibliothek und die Funktionen
- 2.1 Generelle Funktion und Aufruf der Bibliothek
- 2.2 GPS testen und verwenden
- 2.3 GSM testen und verwenden
- 2.4 Die Zustandsmaschine der Bibliothek

3 Beispiel zur Kombination von GSM und GPS
4 Fazit und Ausblick
5 FAQ
6 Einzelnachweise und Downloads

Die Hardware und Anbindung des GPS/GSM Shields

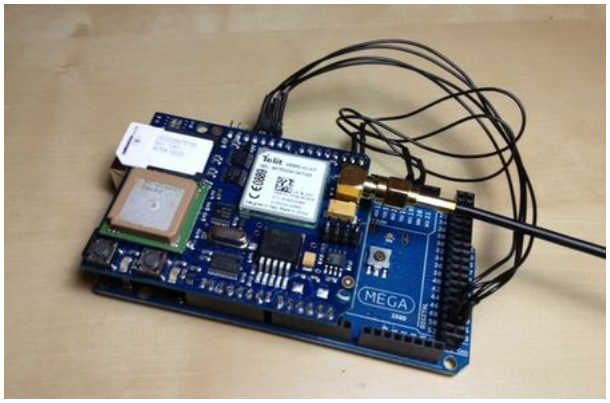
Dieser Abschnitt enthält die wichtigsten technischen Informationen zu den verbauten Komponenten und deren Anbindung. Die Angaben basieren auf den Produktinformationen aus der Seite des Onlineshops[1]. Das Shield ist für einen Arduino Uno ausgelegt, es ist jedoch auch eine Anleitung und sogar ein Umbaukit verfügbar, mit dem die einfache Adaption an einen Arduino Mega 2560 R3 gegeben ist. Die Anbindung des Shields an den Arduino ist mit Level-Shiftern realisiert, d.h. die interne Betriebsspannung und die Pegel des GSM und des GPS Teils können von den üblichen 5V des Arduino abweichen (und tun es auch). Als Level-Shifter Bausteine kommen 74AVC4T774 ICs zum Einsatz. Die notwendigen Betriebsspannungen werden auf dem Shield selbst aus der Ardunio-Spannungsversorgung erzeugt. Für die Verwendung des GSM Moduls wird eine externe Stromversorgung am Arduino empfohlen, da die USB Schnittstelle hier an ihre Grenzen gelangen kann. Ich habe eine Stromaufnahme von 160mA im Leerlauf und 250mA beim senden gemessen. Der notwendige Strom beim senden hängt stark von der Mobilfunk-Abdeckung ab, da soviel Leistung eingesetzt wird wie notwendig ist um sicher mit der nächsten Basis-Station zu kommunizieren.

Verwendete Pins

Die verwendeten Pins sind in der Shieldlist[2] dokumentiert. Aus Gründen der Übersichtlichkeit hier nochmals als Liste (bezogen auf das Arduino Uno Layout).

Arduino Uno Pin	Bedeutung
D0, D1	RX, TX (Serielle Kommunikation mit GSM)
D2	CRING (Serielle Kommunikation mit GSM)
D3	CTS (Serielle Kommunikation mit GSM)
D4	DTR (Serielle Kommunikation mit GSM)
D5	RTS (Serielle Kommunikation mit GSM)
D6	DCD (Serielle Kommunikation mit GSM)
D7	GSM_EN (GSM einschalten)
D9	GPS LED (kann auch via SPI eingeschaltet werden)
D10, D11, D12, D13	SPI Kommunikation mit GPS
A0 als D14	P1 (GSM-Button, kann auch via SPI abgefragt werden)
A1 as D15	P2 (GPS-Button, kann auch via SPI abgefragt werden)

Achtung: Da die RX/TX Pins D0 und D1 verwendet werden, kommt es zu einer Überschneidung mit dem USB-UART eines Arduino Uno (und wenn die Pins nicht anderst verbunden werden auch beim Mega 2560 R3). Das zeigt sich auch daran, dass im Serial Monitor der Arduino IDE die AT Befehle an das GSM Modul zu sehen sind. Siehe dazu auch den Eintrag im Forum [3]. Damit sind die Möglichkeiten etwas limitiert (man kann schwer eine Kommunikation per Serial Monitor mit dem Modul aufbauen), daher wurde für diesen Artikel auf einen Arduino Mega 2560 zurückgegriffen. Für die Verwendung mit dem Arduino Mega 2560 R3 sind die folgenden Pins vorgesehen (Achtung, das bedeutet, das man das Shield nicht einfach stecken kann, es muss statt dessen mit Brücken/Kabeln passend verbunden werden!).



Installiertes Board mit umbelegten Anschlüssen

Das Bild rechts zeigt die Installation auf einem Arduino 2560 R3. Die folgenden Pins sind nicht

direkt mit den Buchsenleisten darunter verbunden, sondern umbelegt (siehe Tabelle unten). Das umbelegen lässt sich an einfachsten realisieren, indem man die entsprechenden Pins am Shield rechtwinklig abbiegt, eine Buchsenleiste mit Kabeln und Silberdraht als Steckern versieht und dann diese seitlich an die abgewinkelten Pins steckt.

Arduino Uno Pin	Bedeutung	Zielpin auf Arduino Mega 2560 R3
0 (RX)	RX des Serial Interfaces gelegt auf Serial 1	19 (RX1)
1 (TX)	TX des Serial Interfaces gelegt auf Serial 1	18 (TX1)
11 SPI MOSI	SPI MOSI	51
12 SPI MISO	SPI MISO	50
13 SPI SCK	SPI SCK	52
14 SPI SS	SPI SS	52

GSM Teil

Es ist das GSM-Modul Telit GE865-QUAD [4] verbaut. Eine Antenne ist nicht auf dem Board, es ist eine externe Antenne notwendig. Diese wird über einen SMA/M-Konnektor angeschlossen. Ebenso ist eine SIM-Karte notwendig (im Mini-SIM Format). Die Betriebsspannung des GSM Teils beträgt 4V. Die Kommunikation erfolgt über Pegelwandler direkt zwischen Arduino-Board und dem GSM Chip. Das Setup der Schnittstelle wird durch die Bibliothek erledigt. Ist die Verbindung hergestellt, werden AT-Kommandos ausgetauscht.

Das Modul unterstützt alle GSM Bänder (daher die Bezeichnung quad für quad-Band). Das Modul hat Features wie einen integrierten TCP/IP Stack, welcher für die GPRS Kommunikation zu einem Webserver verwendet wird. Das Modul kann sogar Anwendungen in einem eingebauten Python Script Interpreter ausführen. Ebenso kann es WAV Dateien lesen und in einem Anruf abspielen. Damit werden Sprachmenüs möglich! Diese fortgeschrittenen Features werden in diesem Artikel allerdings nicht betrachtet.

GPS Teil

Beim GPS Modul handelt es sich um ein UP501 von Fastrax [5]. Die Betriebsspannung des GPS Teils beträgt 3,3V. Die Kommunikation mit dem GPS Modul erfolgt über SPI zum Shield. Verwendet wird SPI Mode 0. Die Geschwindigkeit in der Bibliothek sind 25kHz. Auf dem Shield wird SPI mit einer SC16IS750 SPI-UART Bridge an das eigentliche GPS Modul angebunden. Das bedeutet, dass nach dem Setup der Bridge (wird von der Bibliothek erledigt) über SPI mit dem GPS Modul kommuniziert werden kann, die Fluss-Steuerung erfolgt durch die Kommunikation mit den Registern des SPI-UART Chips. Auch hier sind die Details durch die Bibliothek gekapselt. Die Kommunikation auf Anwendungsebene erfolgt durch den Austauch

von NMEA Sätzen.

Das Modul verfügt über eine integrierte Antenne und eine hohe Empfindlichkeit von -165 dBm. Dabei ist eine Aktualisierungrate von 10Hz möglich. Standardmäßig wird einmal pro Sekunde aktualisiert. Trotz der hohen Empfindlichkeit ist in den meisten Gebäuden kaum der Empfang von ausreichend Daten für eine Positionsbestimmung möglich. Ein Blätterwald sollte jedoch kein Problem darstellen.

Die Software Bibliothek und die Funktionen

Dieser Abschnitt beschreibt die Klassen der Bibliothek, deren Methoden für den Anwender und interne Funktionsweise. Die Bibliothek kommt in einem zip-File, siehe unten der Abschnitt "Downloads". Dieses wird in das "libraries" Verzeichniss von Arduino entpackt. Danach stehen die beiden Headerfiles GSM.h und GPS.h zur Verfügung, die in eigene Programme inkludiert werden können. Ebenso stehen nach einem Neustart der Arduino IDE Beispiele zur Auswahl. Ein Hauptziel bei der Entwicklung der Bibliothek war, dass die Kommunikation mit den Modulen recht offen erfolgt, so dass man nicht durch high-level Funktionen zu sehr abgeschottet ist. Es sollen für die häufigsten Anwendungsfälle aber auch Methoden bereitsgestellt werden, welche die Aufrufe relativ einfach machen.

Generelle Funktion und Aufruf der Bibliothek

Die Bibliothek stellt im wesentlichen zwei Klassen zur Verfügung: GPS und GSM. Dazu kommt noch die Klasse State_Machine, die intern in der GSM Klasse verwendet wird, aber auch für eigene Anwendungen und Erweiterungen benutzt werden kann. Objekte dieser Klassen werden nicht im Headerfile instanziiert. Arduino-Bibliotheken machen das gerne so z.B. die SPI Bibliothek. Darum kann man da gleich SPI.methodeX aufrufen ohne ein Objekt zu instanziieren. Nach dem Instanziieren muss für Objekte der Klassen GSM und GPS einmal die Methode initialize() aufgerufen werden. Diese Methode ist blockierend und kann insbesondere bei der GSM Klasse mehrere Sekunden in Anspruch nehmen. Es bietet sich also an, diese in der setup() Funktion von Arduino aufzurufen. Die Klasse GSM bietet darüber hinaus auch einen Konstruktor an, der mit (1) aufgerufen dafür sorgt, dass Debug-Ausgaben auf das serielle Interface Serial geschrieben werden und am Serial Monitor von Arduino sichtbar sind. Dazu muss aber vor dem Aufruf der initialize-Methode das serielle Interface mit Serial.begin(BAUD) gestartet sein.

```
GPS gps;
// wir wollen Debug-Ausgaben auf Serial.
GSM gsm(1);

void setup() {
  // Serial selbst starten bevor Debug Ausgaben übergeben werden!
  Serial.begin(9600);
  gps.initialize();
  gsm.initialize();
}
```

Basis-Methoden und Komfort-Methoden

Die Methoden der Klassen lassen sich in zwei Arten unterscheiden. Zum einen Basis-Methoden, die recht nahe an der Hardware arbeiten und sich auf den Austausch von Nachrichten beschränken, dabei nicht blockierend sind. Zum anderen Komfort-Methoden, die ganze Abläufe kapseln (beispielsweise den Versand einer SMS), dafür jedoch nicht so tiefe Eingriffe erlauben und blockierend sind, also den Ablauf in der Loop-Schleife für Sekunden unterbrechen können. Ein Hinweis zu den Begriffen: Basis-Methode und Komfort-Methode sind im Software-Engineering nicht präzise definiert und können in der Literatur abweichend verwendet werden.

Aufrufe der Basis-Methoden

Die Basis-Methoden der Klassen sind nicht wartend ausgelegt und nicht blockierend. Das bedeutet, dass Sie gut in der Haupt-Loop aufgerufen werden können, wenn nichts zu tun ist oder noch kein Ergebniss vorliegt kehren Sie sofort zurück. Es bedeutet aber auch, dass die Methoden unter Umständen öfter aufgerufen werden müssen, bis ein Ergebniss vorliegt (weil z.B. ein String noch nicht vollständig vorliegt). Ein Beispiel dazu ist die Methode im folgenden Beispiel (reduziert auf die logischen Schritte, mit pseudocode_ namen markiert). Wichtig: Die initialize Methoden der GPS und GSM Klassen bilden eine Ausnahme, sie kehren erst nach fertiger Initialisierung zurück. Das kann insb. bei der GSM Klasse bis zu 5 Sekunden dauern und sollte daher berücksichtigt werden.

```
bool GPS::read_gps_buffer() {

  pseudocode_lese_zeichen_in_zeile();
  if(pseudocode_zeile_fertig()) {  // <cr> emfpangen?
    return true;
  } else {
    return false;
  }
}
```

Um also alle Zeichen zu lesen und einen kompletten String im Puffer vorzufinden, muss ein Aufrufer die Methode wie folgt benutzen:

```
GPS gps;
void loop() {
  // .... in der main loop andere dinge erledigen
  // prüfen, ob schon ein ganzer String vorliegt
  if(gps.read_gps_buffer()){
    // puffer hält kompletten NMEA String, abholen!
    my_buffer = gps.get_gps_buffer();
  }
  // ....
}
```

Die Methode wird also in jedem Schleifendurchlauf aufgerufen und meldet dann per

Rückgabewert, wenn sie ein sinnvolles Ergebniss erzeugt hat. Wenn das Ergebniss noch nicht da ist, dann vielleicht beim nächsten Aufruf. Das klingt erstmal umständlich, damit hat der Aufrufer aber die bessere Kontrolle über die zeitlichen Ablauf seines Programmes. Oft sollen ja noch andere Dinge erledigt werden, während z.B. das GPS Modul noch kein Ergebniss geliefert hat. Ausflug in die Programmierung allgemein: Wenn eine Funktion nicht in jedem Durchlauf aufgerufen werden soll, dann kann man das gut mit dem Einsatz der millis() Funktion tun. Einfach bei jedem Durchlauf prüfen ob schon genug Zeit vergangen ist. Ein guter Artikel dazu findet sich unter [6]

Aufruf der Komfort-Methoden

Die Komfort-Methoden nutzen intern die Basis-Methoden. Sie sind jedoch so programmiert, dass eine Funktionalität komplett abgearbeitet wird, bevor sie zum Aufrufer zurückkehren. Ein Beispiel ist die Methode send_SMS() der GSM Klasse. Das GSM Modul kann AT Befehle nicht beliebig schnell hintereinander verarbeiten, es sind Wartezeiten notwendig. Um diese Komplexität vom Aufrufer fernzuhalten wird der Ablauf komplett gekapselt. Ein weiteres Beispiel ist die Methode get_next_gps_position der GPS Klasse. Diese blockiert so lange, bis ein vollständiger GPGGA String empfangen wurde und parst diesen dann. In späteren Versionen der Bibliothek sind Versionen dieser Methoden denkbar, die zyklisch aufgerufen werden und dann ihre Fertig-Stellung melden. Für den Artikel wird aus Komplexitätsgründen aber darauf verzichtet. Es wird nämlich auch für den Aufrufer schnell kompliziert seine Aktionen in eine Reihenfolge zu bringen. Ein Aufruf einer Komfort-Methode sieht also so aus (anhand der beiden Beispiele):

```
//...
// diese Methode kehrt erst nach dem Versand zurück. das kann
// ein paar Sekunden dauern.
gsm.send_SMS("0172111222333","Hallo SMS Welt");
//...
nmae_gpgga_message gps_msg;
// warten auf die GPS information als GPGGA String
gps.get_next_gps_position(&gps_msg);
// ...
```

GPS testen und verwenden

Im folgenden Kapitel wird von einem "GPS Fix" gesprochen. "Einen GPS Fix haben" bedeutet, dass das Shield genug Satelliten sieht und genug Daten von diesen empfangen hat, dass es seine Position berechnen kann. Dies vorab für die GPS Neulinge unter den Lesern.

Die GPS Klasse verwendet die SPI Pins der jeweiligen Plattform wie oben in den Tabellen angegeben. Es wird intern die SPI Bibliothek verwendet. Wenn es beim Kompilieren zu Fehlern kommt, kann es helfen im Sketch auch ein #include<SPI> hinzuzufügen - auch wenn dies nicht notwendig sein sollte, da die GPS.h Datei ihrerseits SPI.h includiert. Die Basis-Methoden erlauben das Einlesen von Daten vom Modul via SPI in den Puffer des GPS Objekts und den Zugriff auf diesen Puffer. Die lese-Methode (read_gps_buffer) gibt dabei auch zurück ob das

Puffer eine vollständige Nachricht enthält. Vollständig ist eine Nachricht dann, wenn sie mit <CR><LF> endet. Der öffentliche Teil der Klassendeklaration ist:

```
// ...
// defines fürs einschalten/ausschalten der LED
#define GPS_LED_ON  0x00
#define GPS_LED_OFF 0x07
// defines für die beiden Taster auf der GPS Platine
#define GPS_SWITCH_0    0x00
#define GPS_SWITCH_1    0x01

// eine struktur, die die wichtigsten daten zur positionsbestimmung
enthält.
// wird von parse_nmea_gpgga_string befüllt
typedef struct {
  char  * quality;
  char  * longitude;
  char  * longitude_indicator;
  char  * latitude;
  char  * latitude_indicator;
} nmae_gpgga_message;

class GPS {
  public:
    // gibt true zurück wenn der Buffer eine gültige NMEA nachricht
hat.
    // blockiert nicht.
    bool read_gps_buffer();

    // gibt den buffer zurück (zeiger, KEINE kopie)
    char * get_gps_buffer();

    // blockierend. kehrt zurück wenn ein ganzer GPGGA String
angekommen ist und füllt die info in die Struktur
    void get_next_gps_position(nmae_gpgga_message  * gpgga_msg);

    // parst den puffer auf eine gpgga_nachricht. Verändert
gps_string!
    byte parse_nmea_gpgga_string(char * gps_string, nmae_gpgga_message
* gpgga_msg);

    // initialisiert die Kommunikation. Gibt 1 bei Erfolg zurück.
    char initialize();

    // prüft ob der taster s gedrückt ist
    char check_switch(byte s);

    // schaltet die LED ein oder aus
    void set_LED(char state);

// ....
}
```

Um die rohen Daten des GPS Signals auf dem USB Port auszugeben, kann folgender Beispielcode verwendet werden (bei installierter Bibliothek auch in der IDE verfügbar unter Datei/Beispiele/GSM_GPS/GPS_Test).

```
#include <GPS.h>
#include <SPI.h>

GPS gps;
void setup(){

  Serial.begin(9600);
  if(gps.initialize()){
    Serial.println("Init ok");
  } else {
    Serial.println("Init failure");
  }
}

void loop(){
    // SPI zeichen lesen und prüfen ob damit der String komplett ist
  if(gps.read_gps_buffer()){
    // kompletter string ist vorhanden
    Serial.print(gps.get_gps_buffer());
  }
}
```

Das Programm interpretiert die Daten nicht weiter, sondern prüft immer wieder ob bereits eine vollständige Antwort vom Modul vorliegt. Eine Antwort ist eine Zeile mit <CR><LF> am Ende. Mit diesem Programm ergibt sich bei korrekter Funktion (noch ohne GPS fix) eine Ausgabe wie diese hier

```
Init ok
$GPGGA,134354.765,,,,,0,0,,,M,,M,,*48
$GPGSA,A,1,,,,,,,,,,,,,,,*1E
$GPGSV,1,1,02,05,,,23,26,,,25*7C
$GPRMC,134354.765,V,,,,,0.00,0.00,220213,,,N*4D
```

Wenn das Modul bereits einen GPS Fix hat und die Position bestimmen kann wird die Ausgabe wie folgt aussehen:

```
$GPGGA,193232.000,4844.5529,N,00920.1957,E,1,7,1.06,345.4,M,48.0,M,,*5
8
$GPGSA,A,3,24,09,12,22,26,15,17,,,,,,1.34,1.06,0.83*02
$GPGSV,4,1,13,24,81,324,29,12,48,239,17,09,45,102,30,15,41,183,25*7A
$GPGSV,4,2,13,17,39,064,19,22,15,296,17,18,15,263,,25,14,241,18*72
$GPGSV,4,3,13,14,13,319,,26,09,156,17,28,04,057,,01,02,013,*74
$GPGSV,4,4,13,04,01,109,*46
$GPRMC,193232.000,A,4844.5529,N,00920.1957,E,0.54,151.82,230213,,,A*6F
```

Die wichtigsten NMEA Nachrichten des GPS Moduls

Die NMEA Nachrichten sind gut dokumentiert und werden hier nicht alle besprochen. Grundsätzlich sind die einzelnen Felder der Nachricht durch ',' getrennt und eine Nachricht wird durch <CR><LF> beendet, so dass das Parsen recht einfach ist. Die folgenden Nachrichten sendet das GPS Modul zyklisch:

Nachricht	Inhalt
GPGGA	Positionsdaten und Informationen zur Positionsqualität (letztes Update,....
GPGSA	Informationen zum GPS Fix (2D, 3D, Satelliten und Präzision)
GPGSV	Informationen zu den Satelliten die gesehen werden. Je nach Anzahl der Satelliten können hier mehrere Zeilen auftauchen.

Interpretieren der Positionsdaten mit der Bibliothek

Nach einiger Zeit (bei Wolken und erstmaligem Start nach langer Zeit ohne Strom kann es schonmal 5 Minuten dauern), wird das Shield einen GPS Fix bekommen und kann nun seine Koordinaten berechnen und ausgeben. Das sieht dann aus wie im Beispiel oben (je nach Anzahl der empfangenen Satelliten können mehr Zeilen ausgegeben werden). Von Interesse ist vor allem die Zeile, die mit GPGGA beginnt. Diese Zeile enthält die Koordinaten. Für das Parsen und interpretieren dieser Zeile stellt die Bibliothek eine Methode zur Verfügung. Mit Hilfe dieser Methode wird die GPGGA Zeile so aufbereitet, dass:

1. Ein Programm abfragen kann, ob das Modul seine Position überhaupt feststellen und
2. Die Position aus der Nachricht extrahieren kann.

Für diese Aufgaben stellt die Bibliothek eine Methode und eine Datenstruktur zur Verfügung. Als Datenstruktur dient nmae_gpgga_message, welche die wichtigsten Daten enthält. Befüllt wird sie mit der Methode parse_nmea_gpgga_string.

```
byte parse_nmea_gpgga_string(char * gps_string, nmae_gpgga_message  *
gpgga_msg);

typedef struct {
  char  * quality;
  char  * longitude;
  char  * longitude_indicator;
  char  * latitude;
  char  * latitude_indicator;
} nmae_gpgga_message;
```

Hier ein kleiner Ausflug in den internen Aufbau der Methode, für den interessierten Leser, der sie vielleicht erweitern möchte. Intern nutzt die Bibliothek die String Funktionen der C stdlib. Ähnlich wie die Funktion strtok der stdlib verändert auch die parse-Funktion das Argument

durch das Einfügen von 0-Zeichen. Darum der Hinweis in der Header-Datei. Soll der ursprüngliche String beibehalten werden, muss eine Kopieübergeben werden. Besonders in Verbindung mit der get_gps_buffer Methode ist vorsicht geboten. Diese gibt keine Kopie sondern einen Zeiger auf den internen Puffer zurück. Diese Verhaltensweisen sind absichtlich so gewählt um Speicher zu sparen. Kopien werden nur bei Bedarf von der Anwendung angelegt. Die String-Klasse von Arduino verwende ich nicht, um einigermaßen plattformunabhängig zu bleiben.

Weitere Hinweise zu Stringbearbeitung mit reinem C finden sich z.B. hier im Artikel String-Verarbeitung_in_C. Die Beschreibung der GPGGA Zeile finden wir z.B. unter [7]. Auszugsweise finden wir dort die Information, wie der String aufgebaut ist.

```
$GPGGA,HHMMSS.SS,DDMM.MMMMM,K,DDDMM.MMMMM,L,N,QQ,PP.P,AAAA.AA,M,±XX.XX
,M,SSS,AAAA*CC
```

Dabei sind diese Bestandteile hier die wichtigsten:

Kürzel in String	Bedeutung
N	Qualität der Positionsdaten. • 0 = keine Position • 1,2,3,4,5 = Position vorhanden in unterschiedlichen Qualitäten
DDMM.MMMMM (die erste Gruppe)	Längengrad in Grad, Minuten, Dezimal Minuten
K	Nord oder Süd (N oder S)
DDMM.MMMMM (die zweite Gruppe)	Breitengrad in Grad, Minuten, Dezimal Minuten
L	Ost oder West (E oder W)

Mit diesen Informationen können wir den String untersuchen und die für uns wichtigen Informationen herausfiltern mit der Bibliothek Methode. Das Beispiel ist bei installierter Bibliothek auch zu finden in der IDE unter Datei/Beispiele/GSM_GPS/GPS_Test_parse_gpgga. Eine Besonderheit ist für das Shield zu beachten: Es werden für den Längengrad abweichend von der Spezifikation die Gradzahlen in drei Stellen ausgegeben. Das Format nur für den Längengrad ist bei diesem GPS Modul also DDDMM.MMMMM!

```
#include <GPS.h>
#include <SPI.h>

GPS gps;
void setup(){
  Serial.begin(9600);
  if(gps.initialize()){
```

```
    Serial.println("Init ok");
  } else {
    Serial.println("Init failure");
  }
}

void loop(){
  // SPI zeichen lesen und prüfen ob damit der String komplett ist
  if(gps.read_gps_buffer()){
    nmae_gpgga_message gpgga_msg;

    char * gps_string = gps.get_gps_buffer();

    // weil die parse-funktion den string ändert, kopieren wir ihn
erst
    char buffer[strlen(gps_string)+1]; // trailing 0
    strcpy(buffer,gps_string); // strncopy für produktiven code!

    // wenn der string eine gpgga nachricht ist, bekommen wir die
teile hiermmit und 1 als returncode.
    if(gps.parse_nmea_gpgga_string(buffer,&gpgga_msg)){
      Serial.println(gpgga_msg.quality);
      Serial.print(gpgga_msg.longitude);
      Serial.println(gpgga_msg.longitude_indicator);
      Serial.print(gpgga_msg.latitude);
      Serial.println(gpgga_msg.latitude_indicator);
    }
  }
}
```

Die Ausgabe sieht dann Beispielsweise so aus:

```
Init ok
1
00920.1956E
4844.5529N
1
00920.1953E
4844.5529N
...
```

Oder im Fall, dass noch kein GPS Fix vorhanden ist:

```
Init ok
0
...
```

GPS Daten für Google Maps aufbereiten

Google Maps verwendet für die Java-Script API einen Datentypen namens google.maps.LatLng(int lat, int long). Die Konvertierung erfolgt nach diesen Regeln:

- Bei einem Indikator im Bereich Westen (W) bzw. Süden (S) muss dem entsprechenden Wert ein - vorangestellt werden. Siehe dazu auch [8]
- Google Maps erwartet ein Format der Form DD.MMMMMM.

Es muss also nur das Vorzeichen angepasst werden sowie der Dezimalpunkt verschoben. Dabei bitte die Besonderheit des Längengrads beim GPS Modul auf dem Shield beachten (s.o.). Die Kennzeichnung Ost/West und Nord/Süd fällt weg.

GSM testen und verwenden

Die GSM Klasse verwendet die Serial1 Pins des Arduino Mega 2560 R3. Soll die Anwendung auf einem Uno laufen, muss die Headerdatei GSM.h der Bibliothek entsprechend angepasst werden (#define GSM_SERIAL Serial). Bei eingeschaltetem Debugging werden auf der seriellen Schnittstelle Serial Informaitonen ausgegeben, was gerade passiert. Für die Initialisierung und das anschließende Setup der GPRS Verbindung sieht das Log so aus wie in der angehängten Datei Datei:GSM GPRS Log.txt. Auch diese Klasse stellt die Basis-Methoden zur nicht-blockierenden Kommunikation bereit (lesen und senden vom/an das GSM Modul) sowie einige Komfort-Methoden. Diese kümmern sich nicht nur um den zeitlichen Ablauf komplexer Operationen, sondern auch um Dinge wie Retries bei fehlgeschlagenen Kommandos, Timeout bei wiederholten Versuchen und das Handling eher unschöner Eigenschaften der Kommunikation. Dazu gehört z.B. dass das GSM Modul auch bis zu 2 Sekunden nach der letzten Antwort auf ein AT Kommando nochmals eine Antwort "nachschiebt". Das geschieht beispielsweise beim Setup der GPRS Verbindung, wo auf ein AT+CGAT? zunächste die Antwort "OK" kommt und erst danach CGATT: 0. Nach AT#SGACT kommen zeitversetzt noch 2 "OK" Antworten usw. Bei den Basis-Methoden ist zu beachten, dass die read_gsm_buffer Methode nicht nur Daten vom GSM Modul in den Puffer des GSM Objekts schreibt, sondern diese auch interpretiert. Eine Antwort wird dann als komplett akzeptiert, wenn sie vom GSM Modul mit <CR><LF> beendet wurde. Darum kann diese Methode nicht zum Auslesen von anwendungsspezifischen Daten (etwa nach Anfragen an Web-Server) verwendet werden. Dazu muss direkt GSM_SERIAL verwendet werden.

Der öffentliche Teil der Klassendeklaration ist:

```
// ...
#define GSM_SERIAL Serial1
// ...
class GSM {
  public:
    GSM();
    // wenn mit > 0 aufgerufen, werden Debug informationen nach Serial
    // geschrieben. Achtung. Serial muss vorher mit begin(baud)
gestartet werden
    GSM(byte debug);

    // hier die SIM Pin angeben für die eingelegte Karte.
    // blockierend. braucht 5-8 Sekunden. return 0 bei misserfolg.
    bool initialize(char* sim_pin);
```

```
    // gibt einmal true zurück wenn der buffer eine gültige AW des
modes enthaelt. nicht blockierend.
    bool read_gsm_buffer();

    // achtung, gibt keine kopie zurück sondern pointer auf internen
buffer
    char * get_gsm_buffer();

    // sendet eine SMS an die Nummer mit dem Text. Gibt 1 zurück wenn
ok
    // 0 bei fehler. Blockiert bis Erfolg oder 3 Fehlversuchen
    bool send_SMS(char * phone_number, char * message);

    // gibt den einbuchungsstatus zurück. Darf erst nach initialize
aufgerufen werden
    // 1 = eingebucht heimnetz, 5 eingebucht fremdnetz, andere = nicht
eingeb.
    // blockiert bis eine AW von GSM Modul kommt.
    char get_gsm_state();

    // Bereitet eine Datenübertragung mit GPRS vor.
    // Zugangsdaten des Providers eintragen. Erfordert initialize
vorher.
    bool setup_gprs(char * apn, char * user, char * pwd);

    // sendet die Daten aus message an den Server server auf port port
    // erfordert ein vorher erfolgtes GPRS setup (setup_gprs methode)
    bool send_data_via_gprs(char * server, char * port, char *
message);
//...
}
```

Die initialize-Methode tut das Folgende:

1. Der Enable Pin wird auf Low gezogen
2. Ein Delay von 1000 ms.
3. Der Enable-Pin wird auf High gezogen (default Pin 7, änderbar in header)
4. Ein Delay von 2000 ms (um dem GSM Modul Zeit für Startup zu geben).
5. Das Serial1 Interface wird mit 9600 Baud initialisiert
6. Es werden AT Kommandos abgesetzt um das Modul in einen Zustand zu bringen, in dem es für SMS und Anrufe verwendet werden kann.
    1. AT zum prüfen der Kommunikation
    2. ATE0 zum Abschalten des Echos
    3. AT+IPR=9600 um sicherzustellen, dass das Interface mit 9600 Baud läuft
    4. AT#SIMDET=1 um die SIM Karte zu prüfen
    5. AT+CPIN? um zu prüfen ob eine SIM PIN Eingabe notwendig ist
    6. AT+CPIN=1234(1234 steht für die der Bibliothek übergeben PIN)
    7. AT+CREG? wird aufgerufen um den aktuellen Zustand der Einbuchung zu erfragen. Das ist nützlich vor allem bei der Debug-Ausgabe. Da sich das jederzeit ändern

kann, muss ein Programm dass den Einbuchungsstatus benötigt diesen dann mit get_gsm_state() abfragen.

Diese Schritte werden von einem Zustandsautomaten gesteuert, der festlegt welche Kommandos an das Modul gesendet werden und nach welcher Antwort es wie weitergeht. Diese Schritte vollziehen wir zum besseren Verständniss einmal manuell nach. Dazu dient ein Beispielprogramm, welches auf dem Mega 2560 R3 läuft (denn wir brauchen 2 Serielle Interfaces, eines zum Modul und eines zum PC). Das Beispiel führt nur die Schritte 1 und 2 durch, danach überträgt es Daten von Serial1 nach Serial und umgekehrt. Wir geben die Kommandos dann selbst im Serial Monitor von Arduino ein und schauen das Ergebniss an. Dazu sind folgende Informationen wichtig:

- Das Modul startet mit eingeschaltetem Echo, d.h. wir sehen unser Befehle nochmals in der Konsole.
- Das Modul erwartet nach jedem Kommando <CR><LF>, d.h. wir setzen im Serial Monitor die Einstellung entsprechend (rechts unten auf "sowohl NL als auch CR" setzen.
- Das Modul gibt seine Antwort per Defaul im "extended Mode" zurück. Das bedeutet, wir bekommen immer <CR><LF>Antwort<CR><LF> und damit über und unter der Antwort eine Leerzeile im Serial Monitor.

Das Programm für unser Experiment ist unten gelistet (verfügbar bei installierter Bibliothek in der IDE unter Datei/Beispiele/GSM_GPS/GSM_Test_ohne_lib). Das Zeichen $ verwenden wir um ein CTRL+Z zu senden. Der Serial Monitor kann leider keine nicht-druckbaren Zeichen senden, daher dieser Kunstgriff.

```
// KEIN include von GSM.h, weil wir die Funktion selbst nachbilden
#define GSM_POWER_PIN 7
#define GSM_SERIAL Serial1
#define DEBUG_SERIAL Serial

void setup(){
  GSM_SERIAL.begin(9600);
  DEBUG_SERIAL.begin(9600);

  pinMode(GSM_POWER_PIN,OUTPUT);
  digitalWrite(GSM_POWER_PIN, HIGH);

  delay(2000);
}

void loop(){
  if(GSM_SERIAL.available()){
    DEBUG_SERIAL.write(GSM_SERIAL.read());
  }
  if(DEBUG_SERIAL.available()){
    // handle CTRL+Z (we use $ in serial monitor
    if(DEBUG_SERIAL.peek() == '$')
    {
      char tmp = DEBUG_SERIAL.read();
```

```
      GSM_SERIAL.write(26); // CRTL+Z
    }else{
      GSM_SERIAL.write(DEBUG_SERIAL.read());
    }
  }
}
```

Nach dem Upload und Öffnen des Serial Monitors (und passend einstellen - siehe oben) prüfen wir das Modul. Dabei bezeichnet User> unsere Eingaben und GSM_Modul> die Antworten (<CR><LF> lasse ich raus, daher haben wir weniger Leerzeilen). Details zu den Befehlen, besonders zur SIM Konfiguration und GPRS sowie Anrufen und SMS Versand sind nachzulesen in den Application notes oder z.B. hier[9].

```
User>AT
GSM_Modul>AT
GSM_Modul>OK
```

Echo ausschalten

```
User>ATE0
GSM_Modul>ATE0
GSM_Modul>OK
```

Test ob Echo aus ist

```
User>AT
GSM_Modul>OK
```

Nach diesem Test ist die Kommunikation mit dem Modem sichergestellt.

Eine SMS versenden

> Achtung: Beim Versand von SMS entstehen evtl. Kosten die für die jeweils verwendete SIM Karte berechnet werden!

Für den Versand einer SMS stellt die Bibliothek eine eigene Methode zur Verfügung. Die Methode send_SMS erwartet als Parameter die Ziel-Telefonnummer und den SMS Text (max 240 Zeichen!). Der Versand von SMS wird direkt vom Modul unterstützt und wird mit Hilfe von AT Kommandos erledigt. Es wird der Textmodus gesetzt (SMS unterstützen auch einen Datenmodus, der hier nicht verwendet wird). Bei eingeschaltetem Debugging sind die AT Kommandos und Antworten zu sehen. Diese Methode tut intern das folgende

1. Das Modul wird mit dem Kommando AT auf Verfügbarkeit geprüft
2. Das Modul wird auf den Versand von Text-SMS eingestellt mit AT+CMGF=1
3. Das Modul wird in SMS-Sendebereitschaft versetzt mit AT+CMGS=
4. Der Text wird übergeben und mit CTRL+Z beendet (ASCII Zeichen 26).

Wenn in den Schritten oben eine unerwartete Antwort oder ein Fehler vom GSM Modul gemeldet wird, dann wird der Schritt bis zu drei mal wiederholt, danach wird die Methode abgebrochen. Am Return-Value ist der Erfolg (1) oder ein Problem (0) zu erkennen.

GPRS Verbindung aufbauen und verwenden

> Achtung: Beim Einsatz von GPRS Datenverbindungen entstehen ggfs. Kosten, die entsprechnend der jeweiligen SIM Karte berechnet werden. Besonders wenn kein Datentarif ("Flatrate") abgeschlossen ist, laufen hier schnell erhebliche Kosten auf. Bei diesen Experimenten ist eine Flatrate oder zumindest der Einsatz einer Pre-Paid Karte sehr zu empfehlen!

GRPS ist ein Dienst zur Übertragung von Paket-Daten über das GSM Netz[10]. Dieser Dienst muss vom Netzbetreiber unterstützt werden und es werden Zugangsdaten benötigt. Die Daten umfassen einen sog. APN, einen Benutzernamen und ein Kennwort. Für die meisten Betreiber sind diese Daten nicht abhängig vom einzelnen Kunden sondern fix. Eine Liste ohne Anspruch auf Vollständigkeit ist hier zu finden [11] Diese Daten müssen der Bibliothek mitgeteilt werden (siehe API Funktionen). Die Bibliothek bietet zwei Methoden für die GPRS Kommunikation. Beide sind Komfort-Methoden d.h. sie blockieren bis alle Schritte durchgeführt sind. Die erste Methode setup_gprs führt das Setup durch so dass Verbindungen geöffnet werden können. Die zweite Methode dient der Kommunikation mit einem Webserver. Das Setup muss nur einmal durchgeführt werden, setzt aber eine Einbuchung im Netz voraus.

Die Methoden haben intern den unten beschriebenen Ablauf. Sie verwenden zur Steuerung wie die anderen Methoden eine Zustandsmaschine. Die Setup-Methode tut folgendes:

1. Ein Kontext wird angelegt mit der gegebenen APN und der Anforderung einer dynamischen IP durch das Kommando AT+CGDCONT = 1,"IP","<APN>","0.0.0.0",0,0
2. Die Einbuchung ins GPRS Netz wird mit AT+CGATT=1 erzwungen.
3. Es wird geprüft, ob der Kontext aktiviert werden konnte mit AT+CGATT?
4. Wenn ins GPRS Netz eingebucht wurde (CGATT:1) wird fortgefahren (nach 3 Retries erfolgt ein Abbruch)
5. Der Kontext wird aktiviert mit AT#SGACT = 1, 1, “<USER>”,”<PWD>”

Die Methode zur Kommunikation mit dem Webserver führt intern die folgenden Schritte durch

1. Es wird ein Socket geöffnet auf dem Port/dem Servernamen wie übergeben mit dem Kommando AT#SD=3,0,<Socket>,"<Server_name>"
2. Der übergebene Request wird an den Server gesendet und mit <CR><LF><CR><LF> abgeschlossen. Der Request muss von der Anwendung selbst aufgebaut werden. Eine Hilfestellung findet sich im Beispiel unten. Die Methode kehrt mit true zurück, wenn die Daten an den Webserver gesendet werden konnten. Die Anwendung muss nun selbst die Daten aus dem Puffer lesen und zwar ohne diese als Antworten auf AT Kommandos zu

interpretieren. Das bedeutet, dass die Anwendung auf GSM_SERIAL zugreifen muss (in gsm.h definiert).

Das Beispielprogramm (auch zu finden unter Datei/Beispiele/GSM_GPS /GPRS_google_request) sendet mit Hilfe dieser Methoden eine Anfrage an die Google-Homepage und gibt die HTML Antwort auf Serial aus. Ein Programm, dass die Antwort vom Server verarbeiten will, muss entsprechend das HTTP Protokoll interpretieren.

```
#include <GSM.h>
#include <SPI.h>

GSM gsm(1);
#define SIM_PIN "8263"

byte init_error=0;
void setup(){

  Serial.begin(9600);
  Serial.println("init GSM starting...");
  if(gsm.initialize(SIM_PIN)){
      Serial.println("init GSM ok, setting up GPRS.");
  }else{
      Serial.println("init GSM error!");
      return;
  }

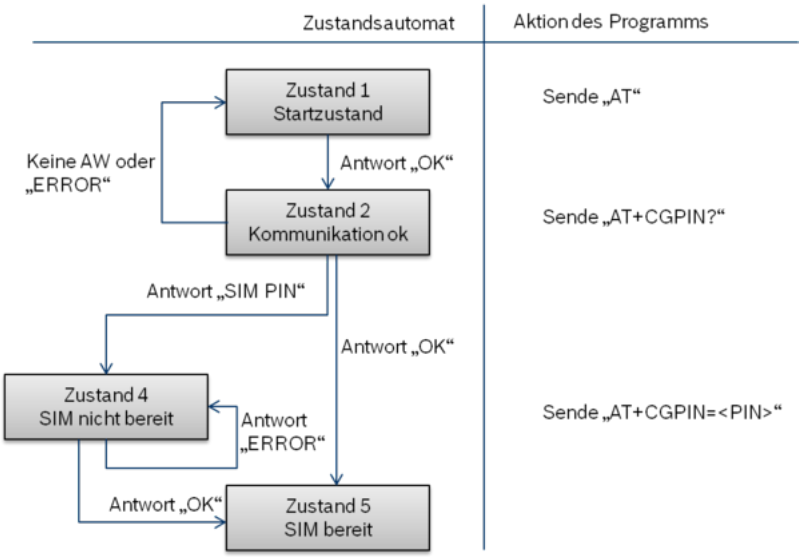
  // EIGENEN APN USER KENNWORT EINTRAGEN
  if(gsm.setup_gprs("internet.eplus.de","eplus","eplus")){
    Serial.println("init GPRS ok.");
   } else {
     Serial.println("init GPRS error!");
    return
   }

  // request senden
  Serial.println("sending HTTP request to google");
  gsm.send_data_via_gprs("www.google.de", "80", "GET /index.html
HTTP/1.1\r\nHost: www.google.de\r\n\r\n");

}

void loop() {
  // direkter Zugriff, da wir keine Interpretation des Puffers wollen
  if(GSM_SERIAL.available()){
    Serial.write(GSM_SERIAL.read());
  }
}
```

Die Zustandsmaschine der Bibliothek



Auszug aus dem Zustandsautomaten der GSM Bibliothek. Zwischenzustände (z.B. Zustand 3) sind entfernt worde um die Übersichtlichkeit zu erhöhen.

Die Bibliothek funktioniert intern für die GSM Abläufe mit einem sog. Zustandsautomaten (engl Statemachine). Dieser ist so konfiguriert, dass je nach aktuellem Zustand und Antwort des GSM Moduls ein neuer Zustand erreicht wird. Es sind Zustandsautomaten definiert für die Initialisierung des Moduls sowie für den Versand einer SMS. Für die Initialisierungsmethode ist dies Auszugsweise im Diagram rechts dargestellt. Sollen eigene Abläufe modelliert werden, kann dies auch mit dem Zustandsautomaten geschehen. Wichtig ist: der Zustandsautomat reagiert mit einer Zustandsänderung auf eingegeben Strings. Das Absetzen des nächsten Kommandos an das GSM Modul ist Aufgabe des Programms. Der Zustandsautomat unterstützt selbst geschriebene Funktionen zum prüfen, ob eine Antworte-Zeile passt. Dies wird über Funktionszeiger realisiert, die beim Setup mitgegeben werden (der letzte Parameter in der add_transition Methode). Dazu sind bereits drei Methoden vordefiniert, die verwendet werden können. Diese Methoden prüfen den Eingangsstring auf einen Treffer am Anfang bzw. am Ende oder geben immer einen Treffer zurück. Das öffentliche Interface der Klasse sieht so aus.

```
//...
// Type eigener Matchingfunktion.
// Gibt != 0 zurück bei Treffer. Erster Parameter ist fixer
Vergleichsstring.
// Zweiter Parameter ist die Zeile, die von aussen übergeben wird.
typedef  byte (*match_func) (char *, char *);
// ...
class State_Machine {
  public:
    // Abfrage des aktuellen Status
    byte get_state();

    // Status manuell setzen
    void set_state(byte state);

    // Die Eingabezeile wird den Zustandsübergängen übergeben. Bei
```

```
einem
    // Treffer (abhängig von der jeweils konfigurierten Funktion und
dem
    // Startzustand) wird der neue Zustand zurückgegeben. 0 bei keinem
Treffer
    byte match_line(char * line);

    // Zustandsübergang hinzufügen. Bei Zustand start_state findet bei
einem
    // erfolgreichen Vergleich von trigger_line mit der Eingabezeile
ein
    // Übergang nach next_state statt. Die Vergleichsfunktion prüft
    // per Default auf gleichen Beginn des Strings. Kann durch eigene
    // Funktionszeiger ersetzt werden (siehe typedef oben).
    void add_transition(byte start_state, byte next_state, char *
trigger_line, match_func f);

    // Vordefinierte Funktionen für matching.
    // Treffer bei gleichem Start
    static byte compare_lines_leading(char * trigger_line, char *
input_line);
    // Treffer bei gleichem Ende
    static byte compare_lines_trailing(char * trigger_line, char *
input_line);
    // Immer Treffer
    static byte compare_lines_allways_match(char * trigger_line, char
* input_line);
//...
```

Verwendung der Zustandsmaschine am Beispiel der initialize Methode des GSM Moduls

Zunächst die Funktion, mit der auf einen Treffer in der AW des Moduls geprüft wird. Standardmäßig wir auf einen Treffer am Anfang des Strings geprüft, wir brauchen aber einen Test auf das Ende des Strings. Daher wird die Methode static byte compare_lines_trailing(char * trigger_line, char * input_line) verwendet. Unten ein Auszug des Bibliotheks-Codes, der die Maschine konfiguriert und dann die Kommandos entsprechend der Zustände absetzt (Achtung, nur zur Demonstration, nicht direkt so ausserhalb der Klasse nutzbar!). Was hier auch zu sehen ist: Da die Zustandsmaschine generisch einsetzbar sein soll, kümmert sie sich nicht um Timings. Die GSM Klasse selbst muss abhängig vom Zustand für Pausen sorgen. Ebenfalls zu sehen ist, wie die GSM Klasse auf Meldungen des GSM Moduls reagiert, bei denen einfach nochmal probiert werden muss. Dies geschieht durch einen Übergang eines Zustands auf sich selbst und eine Erkennung dieses Spezialfalls. Zuerst das Setup der Zustandsmaschine und die Liste der Befehle, die an das GSM Modul gehen. Danach kommen einige Steuervariablen und dann der Teil zum Senden von Kommandos abhängig vom Zustand. Danach die Übergabe der Antwort an die Maschine und die Auswertung der nächsten Aktion (Retry, neues Kommando, keine Aktion).

```
//.....
// befehle und setup der zustandsmaschine
```

```
char * init_commands[] = {"","AT\r","ATE0\r","AT+IPR=9600
\r","AT#SIMDET=1\r","AT+CPIN?\r",pin_setting_command,"AT+CREG?\r"};

init_sm.add_transition(1,2,"OK",State_Machine::compare_lines_trailing)
;

init_sm.add_transition(2,3,"OK",State_Machine::compare_lines_trailing)
;

init_sm.add_transition(3,4,"OK",State_Machine::compare_lines_trailing)
;

init_sm.add_transition(4,5,"OK",State_Machine::compare_lines_trailing)
;
  init_sm.add_transition(5,6,"SIM
PIN",State_Machine::compare_lines_trailing); // PIN nicht gesetzt,
setzen

init_sm.add_transition(5,5,"ERROR",State_Machine::compare_lines_traili
ng); // einfach nochmal versuchen

init_sm.add_transition(5,7,"READY",State_Machine::compare_lines_traili
ng); // Pin schon gesetzt

init_sm.add_transition(6,7,"OK",State_Machine::compare_lines_trailing)
;
  init_sm.set_state(1);

byte last_state = 1;
byte command_sent = 0;
byte retries =0;
byte abort = 0;
// ENDE TEIL 1
```

Jetzt ist die Statemachine bereit und im Startzustand. Die Kommandos, die je nach Zustand an das GSM Modul gehen sind als Array definiert, so dass Sie zu den Zuständen passen. init_commands[X] enthält das Kommando, das in Zustand X an das Modul gesendet wird. Jetzt kann die Maschine in einer Schleife verwendet werden bis der letzte Zustand erreicht ist oder aufgrund von wiederholt fehlgeschlagenen Versuchen abgebrochen wird. Fehlgeschlagen ist ein Kommando dann, wenn die Antwort gar nicht erkannt wird oder wenn die Antwort von Zustand X explizit wieder zu Zustand X führt.

```
// TEIL 2 - hier geht es weiter
while(init_sm.get_state() < 7 && !abort) {

  // sende-teil richtung GSM Modul
  if(command_sent == 0){
    char * command = init_commands[init_sm.get_state()];
    send_command(command);
    command_sent = 1;
  }
```

```
  // leseteil und übergabe an state-machine
  if(read_gsm_buffer()){
    // wir haben eine Antwort bekommen
    // prüfen ob neuer Zustand
    last_state = init_sm.get_state();

    if(init_sm.match_line(get_gsm_buffer())){// Zustandsübergang hat
stattgefunden, match gefunden
      // ist der alte zustand = dem neuen?
      if(last_state == init_sm.get_state()){
        // ja. zuviele retries?
        if(retries++ > MAX_RETRY_AT_COMMAND){
      abort = 1;
        } else {
          delay (500 * retries); // 500ms mal anzahl retries warten
bis neuer versuch
        }
      } else {
        // reset retry auf 0
        retries = 0;
      }
      // im nächsten loop passendes commando senden!
      command_sent = 0;
    } // ende des reads und zustandsübergangs
  }
//.....
```

Mit der Kentniss der internen Programmierung kann nun auch die Ausgabe des Debug-Modus verstanden werden. Die Debug-Aufrufe der Bibliothek sind aus dem Code oben aus Platzgründen entfernt. Mit dem Beispiel unter Datei/Beispiele/GSM_GPS/GSM_Beispiel_mit_lib kann aber der Ablauf nachvollzogen werden. Dort ist der Debug-Modus aktiviert und eine Ausgabe auf der Konsole zeigt folgendes (hier etwas umformatiert):

```
STATE:1
Command:AT
AW:
AW:OK

STATE:2
Command:ATE0
AW:ATE0
AW:OK

STATE:3
Command:AT+IPR=9600
AW:OK

STATE:4
Command:AT#SIMDET=1
AW:OK
```

```
STATE:5
Command:AT+CPIN?
AW:ERROR

STATE:5
Command:AT+CPIN?
AW:ERROR

STATE:5
Command:AT+CPIN?
AW:+CPIN: SIM PIN

STATE:6
Command:AT+CPIN=8263
AW:OK
```

Verwendung der Zustandsmaschine in eigenen Programmen

Für die Verwendung der Zustandsmaschine in eigenen Programmen ist ein Beispiel zu finden unter Datei/Bespiele/GSM_GPS/State_Machine_Demo. Dieses Demo-Programm wartet auf Strings vom seriellen Interface Serial und reagiert dann entsprechend seiner Zustandsmaschine. Es eignet sich gut für eigene Experimente. Der Code ist hier aus Platzgründen nicht gelistet. Mit diesem Code kann man mit dem Serial Monitor nun seine Zustandsmaschine testen. Bitte den Serial Monitor so einstellen, dass ein <CR><LF> gesendet wird (Settings rechts unten). Dann kann ein Test durchgeführt werden, indem Strings an das Programm gesendet werden. Spielen Sie einfach ein wenig damit!

Beispiel zur Kombination von GSM und GPS

Dieses Beispiel bringt die einzelnen Teile nun zusammen. Es wird ein Programm geschrieben, dass auf Tastendruck die aktuellen GPS Koordinaten per SMS an eine Nummer verschickt. Das Beispiel ist vorhanden unter Datei/Beispiele/GSM_GPS/Send_GPS_as_SMS. Es müssen Anpassungen vorgenommen werden (siehe unten). Debugging auf der seriellen Schnittstelle Serial ist eingeschaltet, das kann ausgeschaltet werden, indem der Konstruktor ohne Parameter der Klasse GSM verwendet wird. Im Beispiel wird am Anfang der loop() Schleife die LED auf dem Shield eingeschaltet und am Ende der Schleife wieder aus. Man kann also an den Phase in denen die LED eingeschaltet ist sehen, wenn das Programm in der Schleife beschäftigt ist (eine blockierende Komfort-Funktion aufgerufen ist). Aus Platzgründen ist das Programm hier nicht komplett dargestellt sondern nur besonders interessante Teile. Im oberen Teil des Quellcodes muss die Zielnummer und die PIN der verwendeten SIM Karte eingetragen werden.

```
// ...
// GSM gsm; // ohne Debug-Ausgaben
GSM gsm(1); // mit Debug Ausgaben
// ...
#define PHONE "0172999999" // durch eigene Nummer ersetzen!
#define SIM_PIN "8263" // durch eigene SIM PIN ersetzen
// ....
```

Weiter unten fällt dem Leser vielleicht auf, dass die Abfrage des Tasters nicht explizit entprellt ist. Da wir jedoch Komfort-Funktionen nutzen (wenn die Taste gedrückt wird warten wir zuerst auf eine Rückmeldung vom GPS Modul) und diese eine Verzögerung bewirken, wird dadurch sozusagen implizit entprellt. Es wird keine SMS gesendet, wenn das GPS Modul noch keine Positionsdaten liefern kann. Die Funktion construct_message ist hier nicht dargestellt. Sie bringt die Daten aus der Struktur nmae_gpgga_message in eine gut lesbare Form für den SMS Versand.

```
//.... in der Loop-Funktion
 if(gps.check_switch(GPS_SWITCH_1)){
     Serial.println("Switch pressed");

    nmae_gpgga_message gps_msg;
     // warten auf die GPS information
    gps.get_next_gps_position(&gps_msg);

     Serial.print("GPS Quality:");
     Serial.println(gps_msg.quality);
     // haben wir einen GPS fix?
     if(gps_msg.quality[0] == '0'){
       Serial.println("No GPS Fix yet");
     } else{
       // wir haben einen GPS Fix
       Serial.println("GPS:");
       char sms[240];
       construct_message(sms,gps_msg);
       Serial.println(sms);
       if(gsm.send_SMS(PHONE,sms)){
         Serial.println("Done sending SMS");
       } else {
         Serial.println("ERROR sending SMS");
       }
     }
  }
// ....
```

Fazit und Ausblick

Mit Hilfe des GSM/GPS Shields von Antrax sowie anderer Shields mit ähnlichen Chips können relativ einfach Anwendungen gebaut werden, die sowohl Positionsbestimmung als auch Online-Kommunikation erfordern. Auf einem Arduino Uno ist die Kommunikation mit dem GSM Modul etwas unschön, da sich die Pins mit der USB Kommunikation überschneiden. Für einfaches Debugging und das ausprobieren von AT Kommandos bietet sich daher der Einsatz eines Arduino Mega 2560 an.

Aufbauend auf der Biliothek können schnell eigene, einfache Anwendungen entwickelt werden. Insbesondere die Zustandsmaschine, in Verbindung mit den Methoden die den direkten Datenaustausch mit den Modulen zulassen, bietet eine Basis für eigene Erweiterungen.

Ein möglicher Folgeartikel könnte die Voice-Funktionen des Telit Moduls beschreiben. Die Bibliothek könnte um Methoden ergänzt werden, die noch mehr der internen Funktion kapseln. Für die GPS Bibliothek sind weiter Methoden denkbar, welche die Daten in andere Formen bringen (beispielsweis ins Google-Earth Format oder in eine Form für die Speicherung in KML Files). Die Komfort-Methoden können generell in einer weiteren Variante entwickelt werden so dass auch diese nicht blockieren. Da die Bibliothek frei zugänglich und unter einer Open-Source Lizenz verfügbar ist, sind Erweiterungen und Anpassungen sehr willkommen.

FAQ

Q: Das Shield antwortet nicht auf meine Kommandos. Ich sehe aber Ausgaben im Serial Monitor.
A: Ist der Serial Monitor so eingestellt, dass auch CR und NL gesendet werden? Setting ist rechts unten.

Q: Ich bekomme vom GPS immer eine Quality von 0
A: Es kann besonders zu Beginn eine Weile dauern, bis das Shield einen GPS Fix hat. Wenn sich auch nach 5 Minuten nichts tut, versuche es im Freien. Wenn sich auch da nach 5 Minunten nichts tut könnte ein Fehler vorliegen. Dann würde ich auf den Hersteller zugehen.

Q: Ich kann per GPRS keine Verbindung aufbauen, sondern sehe in den Debug-Ausgaben immer ERROR nach dem Kommando AT#SD=3,0,8080,"x.x.x.x"
A: Prüfe die Daten für APN, User und Kennwort. Diese werden oft zunächst akzeptiert, dann kommt es aber zum Fehler beim Connect.

Q: AT+CGATT=1 ergibt immer einen Fehler (Antwort ERROR).
A: Die Einbuchung ins GPRS Netz funktioniert nicht. Prüfen Sie mit der SIM Karte und einem Handy, ob überhaupt GPRS für die Karte funktioniert und an dem Ort Empfang gegeben ist.

Q: Ich bekomme Compile-Fehler mit Bezug zu SPI, etwa

```
C:\opt\arduino-1.0.3\libraries\GSM_GPS\GPS.cpp:43: error: 'SPI' was
not declared in this scope
C:\opt\arduino-1.0.3\libraries\GSM_GPS\GPS.cpp:43: error: 'SPI_MODE0'
was not declared in this scope
...
```

A: Includiere SPI.h im Sketch. Das Problem kann auch auftauchen wenn nur GSM verwendet wird, da der Arduino-Build immer die ganze Library baut und damit auch den GPS Teil.

Einzelnachweise und Downloads

1. Shield Bezug GSM/GPS Shield im Antrax Online-Shop
2. Antrax GPS/GSM bei Shieldlist Shieldlist
3. Forenbeitrag zum Shield und Arduino UnoForenbeitrag
4. Telit GE865-QUAD Datenblätter und Guides Telit GE865-QUAD Product Page

5. Fastrax UP501 Datenblätter und GuidesFastrax UP501 Product Homepage
6. Artikel zu Arduino und Zeit-Programmierung Tutorial: Arduino und Zeit
7. GPGGA DokumentationGPGGA Dokumentation
8. Latitude und LongitudeLatitude and Longitude
9. GSM AT Kommandos GSM AT Kommandos
10. GPRS Wikipedia zu GPRS
11. GPRS ZugangsdatenGPRS Zugangsdaten für deutsche Provider

Datei:GSM GPS.zip

Kategorien:

- Wettbewerb
- AVR-Projekte
- GPS
- AVR-Boards